

Grundlagen - NeuroEvolution of Augmenting Topologies

Abschlussprojekt — NEAT auf Gymnasium-Benchmarks

Leo Ludwig Thern, Ahmed Abdulrab Mohammed Al-Ghosaini, Saroj Tripathi

13.07.2026

Der Algorithmus

Stanley & Miikkulainen (2002) — **Gewichte und Topologie gemeinsam** evolvieren.

Anders als in Übung 6, wo der EA nur die Gewichte eines *festen* Netzes optimiert hat.

Drei Probleme, drei Antworten:

1. **Wie kreuzt man Netze unterschiedlicher Struktur?** *Innovationsnummern* — jedes neue Gen bekommt eine globale Nummer, Genome werden beim Crossover über diese Nummern ausgerichtet.
2. **Eine neue Topologie ist anfangs schlechter — wie überlebt sie?** *Speziation* — Genome konkurrieren nur **innerhalb** ihrer Spezies, mit Fitness-Sharing. Neue Struktur bekommt eine Schonfrist.
3. **Wie groß soll das Netz sein?** *Komplexifizierung* — minimal starten, Struktur nur wachsen lassen, wenn sie sich auszahlt.

Genom-Repräsentation

- **Node-Gene:** Input / Hidden / Output, jeweils mit Bias
- **Connection-Gene:** in, out, Gewicht, enabled-Flag, **Innovationsnummer**

Das Innovations-Register vergibt pro Lauf für jede Struktur (in, out) genau **eine** Nummer — dieselbe Mutation trägt in verschiedenen Genomen dieselbe Nummer und wird beim Crossover korrekt ausgerichtet.

Strukturelle Mutation

- `add_connection`: neue Verbindung (Zyklus-Check hält das Netz azyklisch)
- `add_node`: splittet eine Verbindung $N1 \rightarrow N2$ in $N1 \rightarrow N_{\text{neu}} \rightarrow N2$

Beim `add_node` bekommt die erste Verbindung Gewicht 1.0, die zweite das alte Gewicht — das neue Neuron ist anfangs **fast wirkungslos** und verschlechtert die Fitness nicht sofort.

Distanz, Crossover, Feed-Forward

Distanz (Grundlage der Speziation): Zahl der Excess- und Disjoint-Gene plus mittlerer Gewichtsunterschied der Matching-Gene.

Crossover: Matching-Gene zufällig von einem Elternteil, Excess- und Disjoint-Gene vom **fitteren** Elternteil.

Netz: aktive Verbindungen werden topologisch sortiert (Kahn-Algorithmus, inspiriert von `feed_forward_layers` aus `neat-python`), dann Knoten der Reihe nach ausgewertet: gewichtete Summe + Bias, danach `tanh`.

Kein Backprop — die Gewichte kommen **allein aus der Evolution**.

Speziation und Reproduktion

- **Fitness-Sharing:** $\text{adjusted_fitness} = \text{fitness} / \text{Speziesgröße}$ — große Spezies ziehen keinen Vorteil aus reiner Größe
- **Nachwuchs** pro Spezies proportional zur summierten adjustierten Fitness
- **Elitismus:** die Besten jeder Spezies überleben unverändert
- **Dynamischer Schwellwert:** wird nachgeregelt, um eine Ziel-Spezieszahl zu halten

Wichtig — negative Fitness: LunarLander und BipedalWalker liefern anfangs negative Rewards. Eine proportionale Aufteilung auf negativen Summen (`negativ / negativ`) würde den Selektionsdruck **umkehren** — die *schlechteren* Spezies bekämen den meisten Nachwuchs.

Deshalb wird die Fitness vor dem Sharing so verschoben, dass sie nicht negativ ist.

Die Fitness ist der **durchschnittliche** akkumulierte Reward über mehrere Episoden.

Warum mehrfach testen? Der Startzustand ist zufällig, ein einzelner Lauf also eine verrauschte Schätzung. Der Mittelwert verhindert, dass “glückliche” Individuen selektiert werden.

Action-Adapter:

- diskret (CartPole, LunarLander) $\rightarrow$ argmax der Ausgaben
- kontinuierlich (BipedalWalker) $\rightarrow$ die $\tanh$ -Ausgaben *sind* die Aktion in $[-1, 1]$

Weiteren auf Jupyter Notebook
